

软件过程改进方案设计文档

1. 改进背景与目标界定

智能交通 OD 预测项目以“区域之间的出行需求”为建模对象，将城市划分为 16×16 网格，以 60 分钟为时间粒度，预测未来 12 个时间步的 OD 矩阵。项目并非单纯追求一个漂亮的模型分数，而是把数据治理、模型训练、过程评估和调度应用连接为一条可审计的软件过程链。

本方案面向该项目的软件过程改进，目标不是把学生项目包装成企业级流程，而是在有限团队、有限算力和有限周期中提升过程的可控性。示范文件采用“过程-目的-预期结果-工作产品”的组织方式，本方案延续这一结构，但将对象从传统 Web 系统转向 AI 驱动的软件项目。

原过程能够支撑模型训练和报告输出，却暴露出三类问题：实验记录分散、数据版本与模型版本绑定不足、评估指标过度依赖平均误差。若不修正，项目后期会出现“结果能展示，却难以解释；模型能运行，却无法复现”的典型风险。

改进目标被限定为课程项目可承受的范围：不新增独立部门，不引入复杂审批，不追求企业级全量流程。方案关注那些一旦缺失就会直接影响提交质量的节点，包括数据基线、实验复现、指标门禁和文档同步。这样的改进比大而全的流程更适合学生团队，也更容易在短周期内看到效果。

2. 原软件过程诊断

原过程以数据采集、数据处理、OD 构建、模型训练、验证评估和可视化发布为主线，结构清晰，却缺少足够的控制点。数据清洗脚本修改后，旧模型是否仍可复现？模型参数调整后，结果表是否同步更新？验证集出现异常波动时，谁负责判断是数据漂移还是训练失败？这些问题没有在流程中被显式回答。

诊断维度	当前表现	风险后果	改进优先级
数据版本	文件命名记录	样本污染难追溯	高
实验管理	手工记录指标	结果表不一致	高
质量门禁	平均 MAE 为主	高峰失败被掩盖	高
文档同步	事后整理	报告与代码脱节	中
风险管理	临时讨论	异常无闭环	中

表 1 原过程诊断结果

诊断结论表明，项目最薄弱处并非编码能力，而是 AI 项目特有的证据链管理。传统软件只要功能可用即可通过部分验收；AI 软件还必须证明结果从何而来、为何可信、在何处失效。

诊断阶段采用问题倒推法：从“报告中哪些结论可能被质疑”反推过程缺口。若教师追问某项指标来自哪个数据版本、哪组参数、哪次训练日志，团队必须能在 3 分钟内定位证据。当前过程尚未完全满足这一要求，因此改进优先级集中在可追溯性和一致性。

3. 改进原则：轻量而可审计

改进方案遵循“轻量而可审计”的原则。轻量意味着不引入庞大的企业级审批，不让团队被模板淹没；可审计意味着每个关键结论都能回到数据版本、模型版本和评估证据。两者看似矛盾，实则互补。

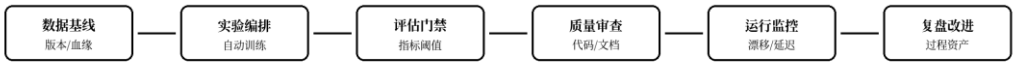
过程改进不追求形式上的完整，而追求风险点的精准控制。对学生团队而言，配置管理计划可以简化为 Git 标签与实验目录规范；质量保证部门可以内化为交叉评审；正式审计可以替换为阶段检查清单。问题不在于是否“像企业”，而在于能否支撑可复现交付。

- 保留关键控制点：需求、数据、训练、评估、发布。
- 压缩低价值文档：不单独编写重型采购、供应和组织级人力资源流程。
- 绑定 AI 证据链：数据版本、模型版本、指标表必须同步归档。
- 设置场景门禁：高峰、异常天气、非零 OD 不再作为附属指标。

轻量化并不意味着降低标准，而是减少无效负担。正式采购流程对本项目价值很低，实验记录却价值极高；厚重的组织管理文档可被剪裁，数据卡片和模型卡片不能省略。过程剪裁的判断依据不是“标准里有没有”，而是“删掉后是否影响质量、复现和验收”。

4. 改进后的总体过程模型

图 3 AI 增强的软件过程改进闭环



改进重点不是“增加工具”，而是把工具嵌入可审计的过程节点。

图 1 AI 增强的软件过程改进闭环

图 1 展示改进后的闭环结构。数据基线确保样本可追溯；实验编排减少人工遗漏；评估门禁阻止不合格模型进入展示版本；质量审查覆盖代码、文档和指标一致性；运行监控捕捉数据漂移；复盘结果沉淀为过程资产。

与原过程相比，改进模型增加了两个关键节点：评估门禁和运行监控。前者保证交付前质量，后者保证交付后仍能发现退化。AI 项目的质量并非在训练完成时结束，它会随着城市出行模式变化而持续波动。

闭环模型把项目从一次性交付拉回持续改进。每轮迭代不再只问模型分数是否提升，还要问提升是否来自有效修改、是否能在固定种子下复现、是否在高峰场景同步改善。若某次修改只改善普通时段却损害异常天气，改进流程会要求团队保留原因分析，而不是简单覆盖旧结果。

5. 数据治理过程改进

数据治理过程被重新定义为独立过程，而不是数据处理脚本的附属环节。每次数据更新必须生成数据卡片，记录来源、时间范围、清洗规则、缺失比例、异常剔除比例和切分方式。示意数据中，原始订单 92.4 万条，清洗后保留 86.8 万条，有效保留率 93.9%；缺失坐标、异常时间和取消订单构成主要剔除项。

工作产品	字段内容	责任角色	验收标准
数据卡片	来源/范围/规模/缺失率	数据负责人	字段完整率 100%
清洗日志	剔除规则/数量	数据负责人	可复跑
切分说明	训练/验证/测试区间	评估负责人	禁止时间泄漏
数据基线	版本号/哈希	项目负责人	与模型版本绑定

表 2 数据治理工作产品

该过程解决的是“数据从哪里来”的问题。没有数据基线，模型提升会失去可信度；一旦训练集与测试集发生时间泄漏，再漂亮的指标也只是幻觉。

数据治理过程还承担团队沟通功能。算法成员往往关注模型结构，文档成员关注结果表，项目负责人关注进度；数据卡片把三者连接到同一事实上。清洗规则、样本数量和切分方式被固定后，讨论才有共同坐标，避免“你用的是哪版数据”成为反复争论的问题。

6. 需求与场景管理改进

需求过程从“描述功能”升级为“描述场景、指标和边界”。例如，“实现 OD 预测”过于宽泛；改进后的需求应写为：系统在工作日早高峰窗口预测未来 12 小时内 16×16 网格 OD 矩阵，并在高流量 OD 上达到不少于 70% 的命中率。

需求编号	场景描述	验收指标	边界条件
REQ-01	常规通勤预测	整体 MAE ≤ 5.2	工作日非节假日
REQ-02	高峰预警	高流量命中 $\geq 70\%$	7-9 点/17-19 点
REQ-03	雨天扰动	雨天 MAE 下降 $\geq 8\%$	小雨到暴雨
REQ-04	可视化展示	延迟 ≤ 2 秒	缓存可用

表 3 场景化需求清单

这样的需求写法减少了后期争议。模型没有必要在所有情况下都完美，但必须明确哪些场景是核心验收对象，哪些场景暂时只做观察。

场景化需求能降低验收模糊性。传统写法中的“提高预测准确率”几乎无法验收，因为准确率可以指整体 MAE，也可以指高峰命中，甚至可以指可视化体验。改进后，每条需求绑定指标、阈值和边界条件，团队在开发前就知道怎样才算完成，答辩时也能更清晰地解释成果。

7. 实验管理与 MLOps 改进

实验管理是 AI 软件过程的核心。方案引入轻量级 MLOps 机制：每次训练生成唯一实验编号，记录代码提交号、数据版本、参数文件、随机种子、训练时长和指标结果。若实验结果写入报告，却无法回溯到配置文件，则该结果不得作为正式交付证据。

该机制并不要求部署复杂平台，课程项目可用 Git + CSV 记录 + 固定目录结构完成。关键不是工具豪华，而是约束稳定。每个实验目录包含 `config.yaml`、`metrics.csv`、`train.log`、`error_cases.md` 和 `model_card.md`，形成最小可审计单元。

- 实验编号格式：EXP-日期-模型-序号，例如 EXP-0707-ODP-03。
- 参数变更必须写入 `config.yaml`，不允许只在代码中临时修改。
- 指标表按整体、高峰、非零 OD、异常天气四层记录。
- 失败实验保留摘要，避免团队重复踩同一类坑。

轻量 MLOps 的核心不是购买平台，而是形成固定证据格式。即使用本地文件夹，只要每次训练都保留配置、日志、指标和误差案例，项目就已经具备基本审计能力。反过来，若没有这些资产，即便使用复杂平台，结果仍可能停留在截图层面，难以支撑正式提交。

8. 质量保证与评估门禁

质量保证过程被设计为“指标门禁 + 交叉评审”。发布版本必须同时满足整体 MAE、高峰 MAE、非零 OD 召回率和推理延迟要求；任何单项不达标，版本只能作为实验候选，不得进入展示包。

门禁项	阈值	检查方式	不通过处理
整体 MAE	≤ 5.20	自动统计	回退上一候选
高峰 MAE	≤ 5.60	分场景评估	补充特征
非零 OD 召回	$\geq 68\%$	稀疏单元统计	调整采样
推理延迟	$\leq 2s$	接口压测	缓存/剪枝
文档一致性	100%	人工评审	修订报告

表 4 评估门禁规则

门禁会带来一个短期不便：某些“看起来不错”的模型可能被挡在发布外。但这恰恰是过程改进的价值。没有门禁，项目容易用单一平均指标掩盖高峰失败；有了门禁，团队必须面对模型的真实边界。

评估门禁会改变团队的行为方式。没有门禁时，成员倾向于展示最高分；有门禁后，成员必须解释每个场景是否达标。质量控制从“结果展示”前移到“候选筛选”，能减少后期返工。尤其在 AI 项目中，门禁不是阻碍创新，而是防止偶然结果被误当成稳定能力。

9. AI 辅助开发与测试改进

AI 工具被纳入开发辅助过程，用于生成测试用例、检查文档一致性、总结训练日志和提示潜在风险。它不能独立批准代码，也不能直接接触真实订单数据。所有输入必须脱敏，所有输出必须由责任人审核。

在接口测试中，AI 可根据 OD 查询接口生成 18 类边界用例：空时间窗口、跨天查询、非法网格编号、无订单区域、极端天气标识缺失等。人工编写通常遗漏的“低频但危险”输入，被系统性补齐。

AI 辅助环节	输入	输出	人工审查点
需求拆解	自然语言需求	用户故事	是否可验收
测试生成	接口字段	边界用例	是否贴合业务
日志摘要	训练日志	失败原因	是否误判
文档校对	报告草稿	不一致项	是否保密

表 5 AI 辅助过程边界

AI 辅助开发的风险主要来自两点：一是输出看似合理却未覆盖业务边界，二是输入材料可能泄露敏感数据。方案把 AI 定位为候选生成器，不把它作为最终裁决者。测试用例、日志摘要和文档修订建议均需人工确认；涉及真实坐标、订单编号和接口密钥的内容必须替换为脱敏样例。

10. 配置管理与文档同步改进

配置管理不再只管理代码，还要管理数据、模型和文档。每个发布包包含四类资产：代码基线、数据基线、模型基线和文档基线。四者缺一不可，因为 AI 项目的结果由它们共同决定。

文档同步采用“指标先行”策略。每次模型结果变更后，`metrics.csv` 自动成为报告中结果表的来源；人工只能解释指标，不能手工改写核心数值。这样可以减少报告与实验记录不一致的问题。

- 代码：Git tag，如 `v2.1-odp-release`。
- 数据：`data_card.md` 与清洗脚本哈希。
- 模型：`model_card.md` 与参数文件。
- 文档：需求表、评估报告、部署说明和风险清单。

配置管理的改进使“版本”从代码概念扩展为系统概念。AI 项目的一个模型文件背后包含数据、特征、参数、随机种子和评价脚本；任何一项变化都可能改变结果。发布包把这些资产绑定后，即使半年后重新打开项目，也能理解当时的技术状态，而不是只剩一个无法解释的权重文件。

11. 过程能力提升评估

图 4 改进前后关键过程能力评分

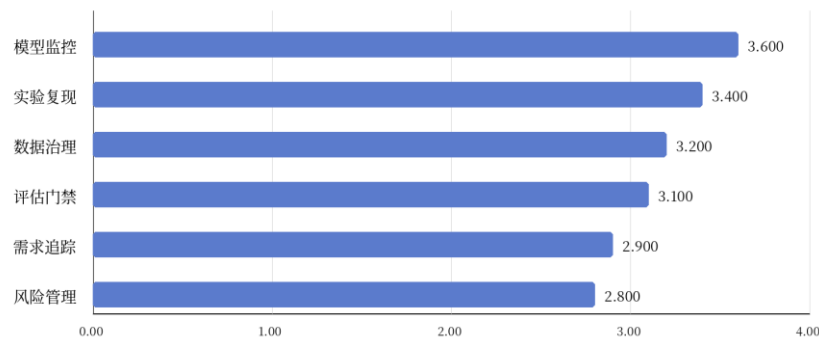


图 2 改进后关键过程能力评分

图 2 给出改进后的能力评分。评分采用 0-4 分制，覆盖模型监控、实验复现、数据治理、评估门禁、需求追踪和风险管理。改进后，实验复现与模型监控提升最明显，因为它们从非正式经验变成了可检查工作产品。

需要注意的是，能力评分不是为了装饰报告。若某项能力连续两轮低于 2.5，项目必须触发过程复盘。例如风险管理得分低，说明问题不是某个模型训练失败，而是风险识别和响应机制缺位。

能力评分用于引导下一轮改进，而不是制造漂亮表格。若模型监控得分提升但风险管理低迷，说明团队看见了指标变化，却没有形成风险响应。若需求追踪得分低，说明场景与代码之间可能脱节。评分本身不解决问题，它通过量化暴露过程薄弱点，促使团队分配有限精力。

12. 风险控制与应急方案

表 2 改进方案中的风险控制矩阵

维度	触发信号	控制措施	责任角色	验收证据
数据泄漏	训练集污染	时间切分	数据负责人	切分日志
概念漂移	分布偏移	漂移检测	算法负责人	PSI报告
指标虚高	高峰欠拟合	场景评估	评估负责人	分层指标
交付延期	迭代阻塞	WIP限制	项目负责人	看板
模型不可解释	黑箱输出	SHAP/案例	算法负责人	解释报告
运行延迟	接口超时	缓存降级	工程负责人	压测记录

表 6 改进方案中的风险控制矩阵

风险矩阵将常见 AI 项目失败点转化为可执行控制措施。数据泄漏通过时间切分与哈希记录控制，概念漂移通过 PSI 和误差趋势监控识别，指标虚高通过场景化评估削弱。

应急策略采用分级响应：轻微指标波动由算法负责人复查配置；连续两次高峰指标不达标则冻结发布候选；数据分布显著漂移时，必须重建数据基线。项目管理不需要把所有风险都消灭，而要在风险出现时知道谁处理、何时处理、凭什么验收。

风险控制矩阵强调“触发信号”。没有触发信号的风险描述只是担忧，难以转化为行动。例如“概念漂移”只有在 PSI 超阈值或连续两周 MAE 升高时才进入处理流程；“指标虚高”只有在分场景结果冲突时触发复查。这样可以避免团队被泛泛风险清单拖慢。

13. 改进落地路径与结论

改进方案按三阶段落地。第 1 周建立数据卡片、实验编号和目录规范；第 2 周引入评估门禁与测试用例生成；第 3 周完成发布包模板和复盘机制。整个方案不依赖昂贵工具，适合课程项目直接执行。

阶段	任务	交付物	验收标准
阶段一	数据与实验基线	data_card/exp_log	可追溯
阶段二	质量门禁与 AI 测试	gate_rules/test_cases	可自动检查
阶段三	发布包与复盘	release_pack/review	可提交

表 7 改进落地路线

本方案的核心判断是：AI 软件过程改进不应被理解为增加更多表格，而应被理解为证据链重构。只有当需求、数据、模型、评估和文档被绑定在同一条链路中，智能交通 OD 预测项目才具备持续迭代能力。

改进路径采用三阶段而非一次性切换，是为了降低组织阻力。先建立数据和实验基线，让团队立刻获得可追溯收益；再加入门禁和 AI 测试，提高质量控制；最后沉淀发布包与复盘模板，形成过程资产。该路线具备渐进性，即便中途时间不足，也能保留最关键的改进成果。